

Quant Developer -- Assignment Submission

Vera Developers -- Placement Process

Grid + Stop-and-Reverse Execution Engine with Bar-Accurate Backtest Harness

Submitted by: Aditya

Chitkara University -- Career Advancement Services referral

Developed with Claude Code, per assignment instructions

1. Scope and Approach

The assignment brief describes a full production trading system: a live grid and stop-and-reverse execution engine, a technical analysis module, a Macro Regime Engine, Zerodha Kite Connect broker/data integration, a backtest harness, order/state management, observability, and automated SDLC agents. Building all of this to production quality is several months of work; within the assignment's timeline the goal was to build a smaller set of pieces to real engineering depth rather than a larger set shallowly.

What was built, fully working and tested:

- Technical analysis module -- one tested implementation per indicator family (trend, momentum, volatility, volume), no duplicated math
- Live grid + stop-and-reverse execution engine -- ATR-based spacing, pyramiding, position caps, portfolio-level kill switch
- Order/state management -- idempotent order placement (deterministic client order ids) and broker-snapshot reconciliation after a simulated crash/restart
- Cost model -- slippage, brokerage, STT/CTT, exchange charges, GST
- Backtest harness -- bar-accurate fills (decision at bar i , fill at bar $i+1$'s open), walk-forward evaluation with per-fold engine state reset
- 39 automated tests, including a crash/restart order-reconciliation regression test

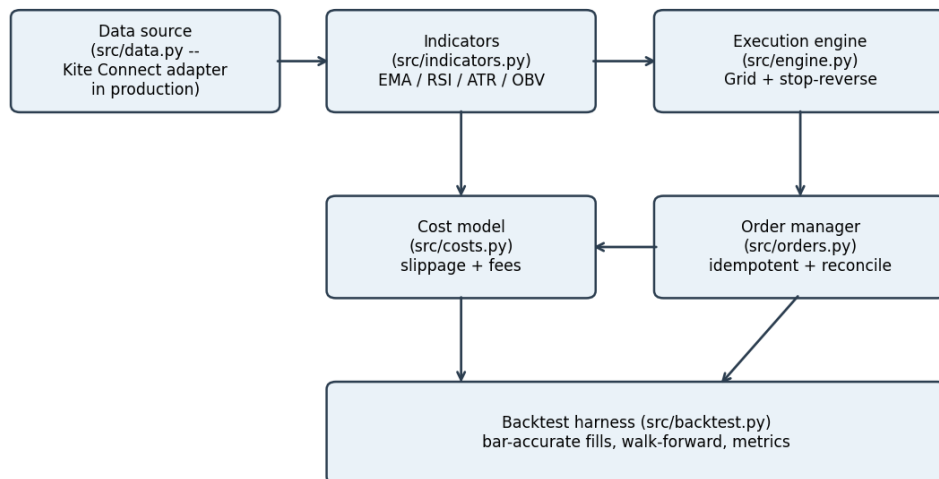
Deliberately deferred, and why:

- Live Zerodha Kite Connect REST/WebSocket integration -- this sandbox has no network route to a broker or market-data API. The data-source module (`src/data.py`) is isolated behind a plain OHLCV DataFrame interface specifically so it is a drop-in replacement point for a Kite Connect adapter, without touching indicators, engine, orders, or the backtester.
- Macro Regime Engine -- scoring macro proxies into regime states and applying parameter overrides is a substantial, largely independent subsystem; given the timeline it was better to leave it out cleanly than implement it as a stub that doesn't actually score anything.
- MCX/NSE contract master and expiry/rollover handling -- requires real instrument data this sandbox cannot fetch.
- An asyncio-based live ingestion loop -- the engine's `on_bar()` call is written so either a synchronous backtest loop or an async live handler can drive it unchanged; only the synchronous (backtest) caller is implemented here.

2. Architecture

The system is deliberately layered so that each concern has exactly one implementation, reused everywhere it's needed:

Grid + Stop-and-Reverse Execution System -- Component Flow



The same `GridReverseEngine.on_bar()` call is what both the backtester and a live handler would drive -- there is only one implementation of the strategy decision logic, which is the mechanism by which backtest numbers are designed to reconcile to live numbers: there is no separate vectorized-backtest version of the strategy that could silently diverge from the live version.

3. Key Design Choices

No lookahead, enforced structurally. Indicators are causal by construction (value at bar *i* uses only data through bar *i*). The backtest harness fills a decision made at bar *i* using bar *i+1*'s open price, never bar *i*'s own close/high/low -- so a signal can never trade at a price that was only knowable at the same instant the signal fired.

Idempotent orders. Every order's `client_order_id` is a deterministic hash of (strategy id, bar index, intent). Replaying the same decision twice -- e.g. after a crash and restart -- is a no-op rather than a duplicate order.

Position truth from fills only. `OrderManager.net_position()` sums only FILLED orders. A pending or rejected order can never inflate the position that risk checks (position cap, kill switch) observe.

Reconciliation as a first-class operation. `OrderManager.reconcile()` takes an external broker snapshot and adopts it as truth for any status mismatch or any order the local process didn't know about -- the same shape a real Kite Connect reconciliation pass after a restart would take.

One cost function everywhere. Slippage and transaction costs are computed by a single `CostModel` used by every fill in the backtester (and would be the same one used to size live orders), so backtest P&L; and a live cost estimate can never silently use different assumptions.

Walk-forward with hard fold isolation. Each walk-forward fold gets its own fresh engine and order manager; no position state or indicator warmup leaks from one fold into the next.

4. Test Coverage

39 automated tests (pytest), organized by module. All pass. Coverage focuses on the properties that actually matter for a trading system, not just line coverage:

Module	Tests	What's checked
indicators.py	10	Hand-checked values, boundary behaviour (RSI 0/100 on monotonic series), causal length, input validation
costs.py	5	Slippage direction, one-sided STT, linear scaling with turnover
orders.py	8	Idempotent client order ids, idempotent placement, fill-only position truth, reconciliation of both status mismatches and unknown orders
engine.py	8	Entry direction, pyramiding trigger, position-cap enforcement, stop-and-reverse trigger and order sequence, kill-switch firing and post-kill halt, idempotent replay
backtest.py	8	Warmup rows produce no signal, fills use next-bar open (not current close), zero-cost fills are exact, position cap holds across a full run, walk-forward windows are chronological/non-overlapping, a crash/restart reconciliation regression test, kill-switch wiring inside the full harness

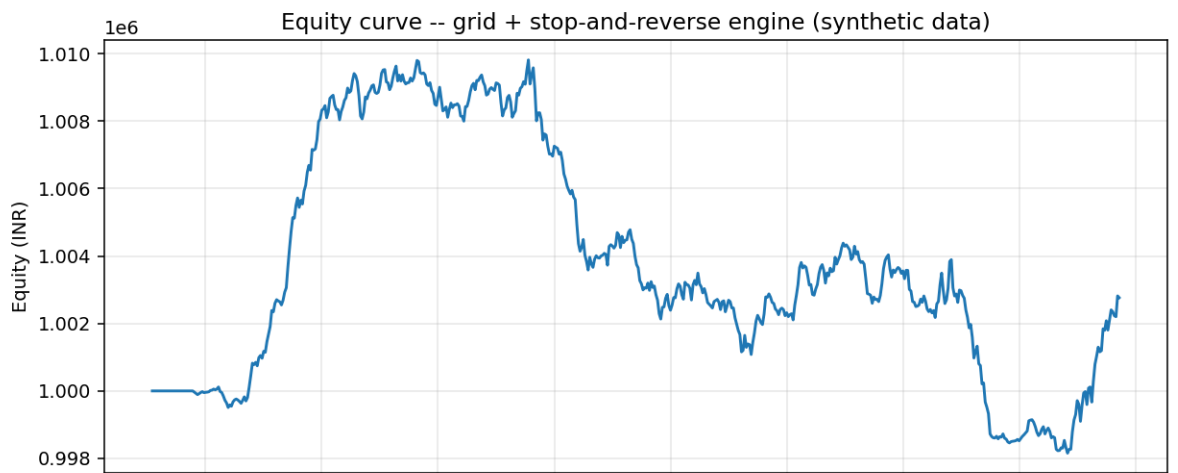
The regression test worth highlighting: it runs a full backtest, snapshots the order book mid-run as "broker truth," then replays the same engine decisions from a completely fresh OrderManager and reconciles against that snapshot -- asserting the resulting net position matches the original run exactly. This is the test that would catch a future change silently breaking crash-safety (e.g. double-submitting orders on restart).

5. Backtest Results

Run on 600 bars (5-minute) of reproducible synthetic OHLCV data with regime-switching trend/chop legs (see `src/data.py` -- this sandbox has no network route to real NSE/MCX data, so results here demonstrate correctness of the mechanics, not claimed alpha). Starting capital INR 10,00,000, contract multiplier 25.

Engine parameter	Value	Cost parameter	Value
ATR multiplier (grid spacing)	1.0	Slippage (bps)	2.0
Stop multiplier	2.5	Brokerage/order (INR)	20.0
Max pyramids	4	STT/CTT (bps, sell)	1.0
Position cap (units)	5.0	Exchange txn (bps)	0.35
Kill-switch drawdown	15%	GST	18%

Metric	Full sample
Trades	18
Total return	0.276%
Max drawdown	-1.155%
Sharpe (annualized, approx.)	2.708
Final equity (INR)	1,002,764.38
Kill switch fired	False



Walk-Forward Evaluation (4 folds)

Each fold uses a fresh engine and order manager (no state or indicator warmup carried across fold boundaries), evaluated in chronological order.

Fold	Bars	Trades	Return %	Max DD %	Sharpe (approx.)
0	180	8	0.016	-0.015	13.243
1	180	12	-0.015	-0.029	-10.777
2	180	21	-0.048	-0.051	-33.015
3	60	5	0.002	-0.008	4.084

Read this as a mechanics check, not a claim of edge: on synthetic regime-switching data with a simple EMA-cross trend filter, results are close to flat after costs in every fold, which is expected -- the point being demonstrated here is that the harness produces stable, reconcilable, cost-aware numbers across independent folds, not that this particular trend filter has alpha on real NSE/MCX data.

6. What's Next

Given more time, the next pieces in priority order would be:

- Kite Connect adapter implementing the same OHLCV-DataFrame interface as `src/data.py`, plus a WebSocket bar-close handler calling the same `GridReverseEngine.on_bar()` used here
- Macro Regime Engine: ingest macro proxies, score into discrete regime states, and apply regime-conditional overrides to `atr_multiplier` / `stop_multiplier` / `position_cap`
- NSE/MCX contract master with expiry/rollover handling, feeding lot-size-aware position sizing
- Structured logging and an alerting blotter (kill-switch fires, reconciliation mismatches) wired to the observability requirements in the brief

7. Repository

Full source, 39 tests, and commit history (7 commits, staged by module: indicators -> costs -> orders -> engine -> data -> backtest harness -> run script/README) are included alongside this report. See README.md for exact run instructions (``pytest tests/ -v``, ``python run_backtest.py``).